RELATIONAL
TEMPORAL

音の構造的関係を、時間とピッチの両軸に同時転写する原理

Iceface | Theory & Implementation | 2026

WHAT IS RELATIONAL?

「関係的」とは何か

従来: 絶対的 (Absolute)

各音は固定された値を持つ

ピッチ: C# = 100¢（常に固定）
リズム: 全ての音が等間隔に並ぶ

音と音の間に構造的な関係性がない
12-TET / 従来のアルペジエーター

Iceface: 関係的 (Relational)

各音は隣接する構造音との関係で決まる

ピッチ: 黒鍵 = 右隣白鍵との距離 N で定義
リズム: 黒鍵 = 白鍵との間隔比 N で定義

N という単一パラメータが全ての非ダイアトニック音の振る舞いを制御する

核心原理: 音楽において「白鍵」は構造的な参照点（diatonic reference）であり、
「黒鍵」はその参照点との **関係（距離 N）** によって定義される。
この原理はピッチだけでなく、あらゆる音楽パラメータに適用できる。

STRUCTURAL REFERENCE

D — ダイアトニック参照位置

C	C#	D	D#	E	F	F#	G	G#	A	A#	B
D	非D	D	非D	D	D	非D	D	非D	D	非D	D

D = { 0, 2, 4, 5, 7, 9, 11 }　　非D = { 1, 3, 6, 8, 10 }

D の意味 (IS)

- ✓ ピアノ鍵盤の白鍵の位置
- ✓ keyboard-topological（鍵盤位相的）
- ✓ 物理的・構造的な事実
- ✓ **全ての Iceface 変換の基盤**

D が意味しないもの (IS NOT)

- × 調性的スケールディグリー（I, II, III...）
- × 機能和声における度数
- × 特定の調（key）に依存する音名
- × 音楽理論上のダイアトニックスケール

RELATIONAL-TONAL

I(D,N) — Iceface Tuning

黒鍵のピッチ = 右隣の白鍵 − N cents

ピッチ空間に Relational 原理を適用した最初の形態。
Iceface Toy One で実装。白鍵のピッチは固定し、黒鍵のみが N で動く。

N = 0¢

ダイアトニック純粋

黒鍵 = 右隣白鍵
（同音 → 消滅）
C D E F G A B のみ

N = 50¢

Iceface Tuning

黒鍵が右隣白鍵方向に
50¢引き寄せられる
独自の微分音チューニング

N = 100¢

12-TET（従来）

黒鍵 = 通常の半音
標準的な平均律
すべての音が等距離

RELATIONAL-TEMPORAL

T(D,N) — Dual Transformation

I(D,N) がピッチ空間で行った変換を、時間空間にも同時に転写したもの。
IcefaceTDN の KINETIC モードで実現。

時間軸: 黒鍵のステップ間隔 = stepSize × N

ピッチ軸: 黒鍵のオフセット = +(1 − N) × 100¢

N = 0

純粋ダイアトニック

時間: ステップ = 0
→ 非D音を完全スキップ

ピッチ: オフセット = +100¢
→ 右隣白鍵と同音（消滅）

C D E F G A B が等間隔で鳴る

N = 0.5

Iceface 非対称

時間: ステップ = step × 0.5
→ 白鍵の半分の間隔で発音

ピッチ: オフセット = +50¢
→ 右隣白鍵に50¢近づく

I(D,N) の時空間的完全再現

N = 1.0

等間隔（従来）

時間: ステップ = step × 1.0
→ 白鍵と同じ間隔

ピッチ: オフセット = 0¢
→ 12-TET そのまま

従来のクロマティックアルペジエーター

WHY DUAL?

なぜ時間だけでなくピッチも変換するのか

temporal-only（不完全）

時間間隔だけを N で変える場合:

N = 0.5 のとき:
C# は C の半分の間隔で早く鳴るが、
ピッチは 12-TET のまま（100¢）

問題: I(D,N) との対称性が崩れる。
Iceface Tuning は「黒鍵を白鍵に引き寄せる」発想だが、
temporal-only では
時間だけ寄せてピッチは遠いまま。

dual transformation（完全）

時間 + ピッチを同時に N で変える:

N = 0.5 のとき:
C# は C の半分の間隔で早く鳴り、
かつピッチも D 方向に 50¢ 近づく

結果: I(D,N) との完全な対称が実現。
黒鍵は時間的にも音高的にも白鍵に「引き寄せられる」。
N = 0 で両軸とも消滅 → 完全一貫。

I(D,N): ピッチ = 右隣白鍵 − N¢
T(D,N): ステップ = step × N, ピッチ += (1−N) × 100¢
→ N が 0 に向かうと、黒鍵は時間的にもピッチ的にも白鍵に吸収される

IMPLEMENTATION

PlaybackEngine.tickKinetic() の実装

時間軸変換

```
// 現在の音のダイアトニック判定
let note = noteArray[kineticSeqIndex]
let isDia = D.contains(note % 12)

// ステップ間隔の決定
if isDia:
  step = kStep          // 基準
else if N ≈ 0:
  advanceIndex()        // スキップ
else:
  step = kStep × N      // N で縮小

// 移動距離がステップを超えたら発音
if accum >= step → trigger
```

ピッチ軸変換

```
// 発音時にピッチオフセットを適用
let pc = note % 12
let isDia = D.contains(pc)

let cents =
  isDia ? 0.0              // 白鍵は不変
       : (1-N) × 100.0     // 黒鍵を偏移

// 内蔵シンセ: 直接マイクロチューニング
audio.playNote(note, cents)

// MIDI出力: ピッチベンドで表現
midi.pitchBend(8192 + cents*40.96)
```

注: current-note semantics を採るため、RANDOM direction でも追加の peek なしで同じ規則を適用できる

UNIFIED PRINCIPLE

統一原理

ダイアトニック参照位置 D に対して、
非ダイアトニック音の振る舞いを **単一パラメータ N** で制御する。
この原理は、適用先を問わない。

ピッチ空間	**I(D,N)**	Iceface Toy One	`pitch = neighbor − N¢`
時間+ピッチ空間	**T(D,N)**	IcefaceTDN KINETIC	`step = s×N,  pitch += (1−N)×100¢`
（将来の拡張）	**?**	velocity, timbre…	同じ D と N で任意のパラメータへ

Relational — 構造的関係を、音楽のあらゆる次元に転写する原理